

Object-Oriented Programming (OOP)

Comprehensive Architectural Principles, Core Pillars & SOLID Design Best Practices

Object-Oriented Programming (OOP) is a programming paradigm centered around *objects*—data structures containing fields (attributes/properties) and code (methods/functions). OOP models software design around real-world entities to maximize modularity, reusability, maintainability, and code organization.

1. Fundamental Building Blocks: Class vs. Object

- **Class:** A user-defined blueprint or template that defines the structure (attributes) and behaviors (methods) that objects created from it will possess.
- **Object:** A concrete, runtime instance of a class loaded into memory with real values assigned to its fields.

```
// Blueprint (Class)
public class Car
{
    public string Model { get; set; }
    public int Speed { get; set; }

    public void Accelerate(int increment) => Speed += increment;
}

// Runtime Instance (Object)
Car myCar = new Car { Model = "Tesla Model 3", Speed = 0 };
myCar.Accelerate(60); // Speed is now 60
```

2. The 4 Main Pillars of OOP

I. Encapsulation

Bundling data and methods into a single unit while restricting direct access to internal state via access modifiers (`private` , `protected` , `public`). Ensures internal consistency and prevents invalid state modifications.

```
public class BankAccount
{
    private decimal _balance; // Hidden internal state

    public decimal GetBalance() => _balance;

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Amount must be positive.");
        _balance += amount;
    }
}
```

II. Abstraction

Hiding complex background implementation details and exposing only the essential interface needed to interact with the object (achieved using `abstract` classes or `interface` contracts).

```
public interface IPaymentProcessor
{
    bool ProcessPayment(decimal amount); // Exposes 'what', hides 'how'
}

public class CreditCardProcessor : IPaymentProcessor
{
    public bool ProcessPayment(decimal amount)
    {
        // Complex network encryption and gateway handshake logic hidden here
        return true;
    }
}
```

```
}  
}
```

III. Inheritance

Mechanism where a child/derived class inherits attributes and methods from a parent/base class, enabling code reuse and hierarchical relationships.

```
public class Vehicle // Parent Base Class  
{  
    public string Brand { get; set; }  
    public void StartEngine() => Console.WriteLine("Engine started.");  
}  
  
public class ElectricCar : Vehicle // Child Derived Class  
{  
    public int BatteryCapacity { get; set; } // Adds specific attributes  
}
```

IV. Polymorphism

Ability for an entity to take multiple forms ("many shapes"). Allows treating objects of different derived classes through a common base class or interface.

- **Compile-time (Static / Overloading):** Multiple methods in the same class with identical names but different signatures/parameter lists.
- **Runtime (Dynamic / Overriding):** Derived class redefines virtual/abstract methods defined in a parent class.

```
public abstract class Animal  
{  
    public abstract void MakeSound(); // Dynamic Polymorphism target  
}  
  
public class Dog : Animal { public override void MakeSound() => Console.WriteLine("Bark!"); }  
public class Cat : Animal { public override void MakeSound() => Console.WriteLine("Meow!"); }  
  
// Poly usage: List<Animal> animals = [new Dog(), new Cat()];  
// foreach (var a in animals) a.MakeSound(); // Executes respective implementations
```

3. SOLID Architectural Principles

Principle	Description
Single Responsibility (SRP)	A class should have one, and only one, reason to change.
Open/Closed (OCP)	Software entities should be open for extension, but closed for modification.
Liskov Substitution (LSP)	Subtypes must be completely substitutable for their base types without altering correctness.
Interface Segregation (ISP)	Clients should not be forced to depend on interfaces they do not use (prefer small, targeted interfaces).
Dependency Inversion (DIP)	High-level modules should depend on abstractions, not on low-level concrete implementations.